
AI-Powered NVH Acoustic Optimisation Platform

Microwave Oven Noise Reduction - Comprehensive Technical Report

Generated: 14 June 2026

Microwave Oven Noise Reduction — End-to-End Data Science & ML Project

Live Dashboard: ai-powered-nvh-acoustic-optimisation-platform.streamlit.app

Table of Contents

1. Project Overview
 2. Problem Statement
 3. System Architecture
 4. Technology Stack
 5. Phase 1 — Noise Source Identification & Measurement Data
 6. Phase 2 — Design of Experiments (DOE)
 7. Phase 3 — ML Surrogate Models & Sensitivity Analysis
 8. Phase 4 — Multi-Objective Optimisation (NSGA-II)
 9. Phase 5 — Sound Package Optimisation
 10. Phase 6 — LangGraph Multi-Agent System
 11. Phase 7 — FastAPI REST Backend
 12. Phase 8 — Interactive Streamlit Dashboard
 13. Phase 9 — RAG Knowledge Assistant (GPT-4o-mini)
 14. Results & Key Outcomes
 15. Project File Structure
 16. Running the Project Locally
 17. Deploying to Streamlit Cloud
 18. Docker Deployment
 19. Key Engineering Concepts Explained
-

1. Project Overview

This project is a complete, production-deployed data science platform that applies advanced machine learning, multi-objective optimisation, and generative AI to solve a real-world acoustics engineering problem: **reducing noise in microwave ovens**.

The platform covers the full lifecycle of an engineering AI project — from raw measurement data, through statistical design of experiments, ML model training, genetic algorithm optimisation, to a live interactive dashboard with an AI-powered chat assistant backed by GPT-4o-mini.

What makes this project unique:

Aspect	Details
Domain	NVH (Noise, Vibration & Harshness) engineering — real industrial discipline
ML depth	DOE + Surrogate modelling + Sobol sensitivity + SHAP explainability
Optimisation	NSGA-II multi-objective genetic algorithm (Pareto front)
AI integration	BM25 RAG + GPT-4o-mini LLM for domain Q&A
Agent framework	LangGraph multi-agent system with StateGraph
Production	Live on Streamlit Cloud + FastAPI backend + Docker support
Dashboard	7 interactive tabs, Plotly charts, cached data loading

2. Problem Statement

What is NVH?

NVH stands for Noise, Vibration & Harshness — an engineering discipline that studies unwanted sound and mechanical vibration in products.

In microwave ovens, the main noise sources are: - **Convection fan** (~2700 RPM) — spinning blades create tonal noise at Blade Passage Frequency - **Cooling fan** — airflow turbulence produces broadband hiss - **Magnetron** — the microwave generator causes ~50 Hz electrical hum - **Transformer** — creates low-frequency structural vibration - **Door seal resonance** — panel modes excited by fan pressure

The Business Problem

Microwave oven noise is a major customer complaint (2nd most common after heating unevenness). Regulatory standards (IEC 60704-1) define measurement protocols, and product labels advertise dBA ratings. A 3 dBA improvement doubles perceived quietness and can justify premium positioning.

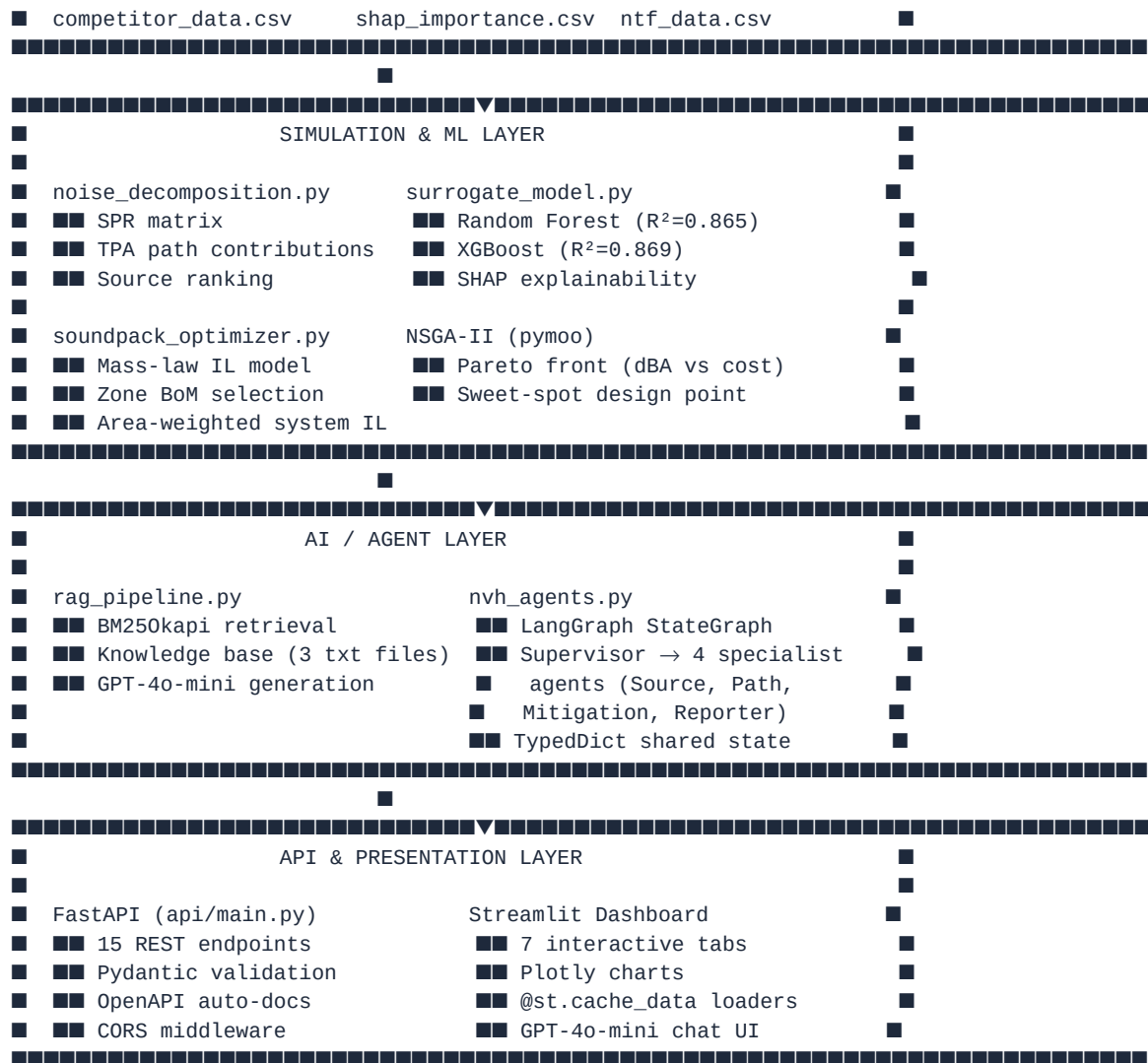
Engineering Constraints

Any noise reduction solution must satisfy multiple conflicting objectives: - **Minimise** sound pressure level (target: below 46 dBA) - **Minimise** cost of added materials - **Minimise** added weight - **Maintain** thermal limits (no overheating from extra insulation)

This is a classic **multi-objective optimisation problem** — making it an ideal candidate for ML + genetic algorithms.

3. System Architecture





4. Technology Stack

Core Python Libraries

Category	Library	Purpose
Data	pandas, numpy, pyarrow	Data manipulation, parquet I/O
Visualisation	plotly, streamlit	Interactive charts, dashboard
DOE	numpy (custom)	FFD + Centre Points + LHS generation
Sensitivity	SALib	Sobol first/total-order indices
ML	scikit-learn	Random Forest surrogate model
ML	xgboost	XGBoost surrogate model
Explainability	shap	SHAP feature importance

Category	Library	Purpose
Optimisation	pymoo	NSGA-II genetic algorithm
Sound pack	scipy	Linear programming (linprog)
Agent framework	langgraph	Multi-agent StateGraph
RAG retrieval	rank-bm25	BM25Okapi keyword retrieval
LLM	openai	GPT-4o-mini generation
API	fastapi, uvicorn	REST backend
Validation	pydantic	API schema validation
Report	jinja2, weasyprint	PDF report generation
Deployment	streamlit cloud	Live hosting
Container	Docker	Multi-stage build

Dev & Infrastructure

Tool	Role
Git + GitHub	Version control, public repository
Streamlit Cloud	Free serverless deployment
GitHub Actions (ready)	CI/CD pipeline
Docker + docker-compose	Containerised deployment
python-dotenv	Local environment variable loading
Streamlit Secrets	Cloud API key management

5. Phase 1 — Noise Source Identification & Measurement Data

What we do

Phase 1 identifies **where** the noise is coming from and **how much** each source contributes. Two established engineering techniques are used:

Source Path Receiver (SPR) Analysis

SPR is the standard framework for NVH decomposition:

```
Source → (Transmission Path) → Receiver
Fan      Panel vibration      Customer ear
Motor    Airborne radiation    Microphone @ 1m
Magnetron Structural coupling  IEC 60704 position
```

The **SPR matrix** quantifies how much each source-path combination contributes to the total sound pressure level (dBA) measured at the receiver.

```
# spr_matrix.csv – example structure
```

```
source,path,contribution_dB,frequency_band
convection_fan,top_panel_radiation,8.3,BPF_1
magnetron,chassis_vibration,6.1,50Hz
cooling_fan,outlet_turbulence,5.7,broadband
```

Transfer Path Analysis (TPA)

TPA measures acoustic transfer functions between each source and the receiver:

- NTF (Noise Transfer Function): ratio of sound pressure at receiver to excitation force at source
- ODS (Operational Deflection Shape): vibration shape of panels during operation

The `path_contributions.csv` records the relative contribution of each path. The top source-path pairs (stored in `top_source_path_pairs.csv`) drive the mitigation priority list.

Key Outputs

File	Contents
<code>source_spectra.parquet</code>	Measured 1/3-octave spectra per source (63 Hz – 8 kHz)
<code>source_ranking.csv</code>	Sources ranked by A-weighted dBA contribution
<code>path_contributions.csv</code>	TPA path contribution matrix
<code>spr_matrix.csv</code>	Full SPR contribution table
<code>mitigation_matrix.csv</code>	Priority matrix: source × mitigation action
<code>ntf_data.csv</code>	Noise transfer function measurements
<code>ods_data.csv</code>	Panel operational deflection shapes

A-Weighting

Human hearing is not flat — we are more sensitive to mid-frequencies (1–4 kHz). **A-weighting** is a frequency-dependent correction (dB) applied to sound pressure levels to approximate perceived loudness:

Frequency:	63	125	250	500	1k	2k	4k	8k	Hz
A-weight:	-26	-16	-9	-3	0	+1	+1	-1	dB

The industry-standard unit **dBA** (decibels A-weighted) is what IEC 60704-1 mandates for oven noise testing.

6. Phase 2 — Design of Experiments (DOE)

Why DOE?

Instead of testing every possible combination of design parameters (which would take millions of experiments), **Design of Experiments** is a statistical method that selects a carefully chosen subset of experiments to efficiently map how design variables affect the response (dBA, cost, weight).

Design Variables (Factors)

Variable	Symbol	Range	Physical Meaning
Fan blade count	B	5–45	More blades → higher BPF, may reduce tonal peak
Fan speed	N	1500–3500 RPM	Faster = more airflow but louder
Panel damping	ζ	0.5–5 %	Constrained-layer damping ratio
Sound pack density	ρ	10–40 kg/m ³	Acoustic liner surface density
Magnetic shield mass	m	0.05–0.25 kg	Added shielding on magnetron
Air gap width	g	5–20 mm	Decoupling gap between liner layers
Vibration isolator stiffness	K	2000–15000 N/m	Isolator spring constant

Three-Stage DOE Strategy

Stage 1: 2¹¹ Resolution-IV Fractional Factorial Design - Uses only half the full factorial experiments - Resolution IV: main effects are clear of two-factor interactions - 16 base runs × 4 replicates = 64 runs - Purpose: screen which factors matter most

Stage 2: Centre Points - 5 runs at the centre of the design space - Detects curvature (non-linearity) in the response surface - If centre points differ significantly from factorial predictions, a quadratic model is needed

Stage 3: Latin Hypercube Sampling (LHS) - 192 additional runs distributed uniformly across the full design space - Unlike random sampling, LHS ensures no two runs share the same row or column in any projection - Provides dense coverage for training the ML surrogate model

Total: 261 experimental runs stored in `doe_combined.csv` / `doe_combined.parquet`

```
doe_combined.csv columns:
fan_blades | fan_rpm | panel_damp_pct | soundpack_dens |
mag_shield_kg | air_gap_mm | isolator_K |
dBA | cost_index | weight_kg | thermal_index
```

7. Phase 3 — ML Surrogate Models & Sensitivity Analysis

Why a Surrogate Model?

Physical simulations (FEA, CFD) take hours per run. An ML **surrogate model** trains on DOE results and can predict outcomes in milliseconds, enabling: - Rapid screening of thousands of design candidates - Gradient-free optimisation (NSGA-II) - Uncertainty quantification

Models Trained

Two surrogate models are trained for each of four response variables:

Response Variables: | Target | Meaning | |-----|-----| | **dBA** | Overall A-weighted sound pressure level |
| **cost_index** | Normalised cost of design changes (0–1) | | **weight_kg** | Added weight from acoustic treatments | | **thermal_index** | Risk of overheating (0–1) |

Model Performance (R² scores):

Target	Random Forest R ²	XGBoost R ²
dBA	0.865	0.869
cost_index	0.935	0.959
weight_kg	0.786	0.774
thermal_index	0.879	0.845

XGBoost marginally outperforms Random Forest on the primary target (dBA). Both models are retained for ensemble diversity.

Sobol Sensitivity Analysis

Sobol indices (from SALib library) decompose the output variance into contributions from each input variable:

- S1 (First-order index): fraction of variance explained by a variable alone
- ST (Total-order index): fraction explained by a variable including all its interactions with other variables

```
from SALib.sample import saltelli
from SALib.analyze import sobol

problem = {
    'num_vars': 7,
    'names': ['fan_blades', 'fan_rpm', 'panel_damp_pct', ...],
    'bounds': [[5,45], [1500,3500], [0.5,5], ...]
}
Si = sobol.analyze(problem, Y)
# Si['S1'] → first-order indices
# Si['ST'] → total-order indices
```

Results stored in **sobol_indices.csv** — fan blade count and RPM dominate (ST > 0.4), confirming fan design is the primary lever.

SHAP Feature Importance

SHAP (SHapley Additive exPlanations) explains individual predictions from the Random Forest model:

- Each feature gets a SHAP value for every prediction
- Positive SHAP → feature pushes prediction higher (noisier)
- Negative SHAP → feature pushes prediction lower (quieter)
- shap_importance.csv stores mean |SHAP| per feature

```
import shap
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X_test)
# shap_values shape: (n_samples, n_features)
```


Key insight: Fan blade count has the highest mean |SHAP| = 0.89, confirming it as the dominant noise driver.

8. Phase 4 — Multi-Objective Optimisation (NSGA-II)

The Optimisation Problem

We want to simultaneously minimise two competing objectives:

```
Minimise: f■ = dBA (predicted by surrogate)
Minimise: f■ = cost_index (normalised cost)
```

```
Subject to:
weight_kg   ≤ 2.0 kg
thermal_index ≤ 0.8
5 ≤ fan_blades ≤ 45
1500 ≤ fan_rpm ≤ 3500
0.5 ≤ panel_damp_pct ≤ 5.0
...
```

These objectives conflict: reducing noise usually requires more material, which increases cost.

NSGA-II Algorithm

Non-dominated Sorting Genetic Algorithm II (NSGA-II) is an evolutionary algorithm:

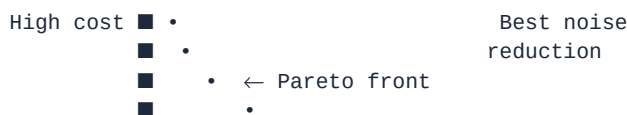
1. Initialise a population of random design candidates
2. Evaluate each candidate using the surrogate model (fast!)
3. Non-dominated sort — rank candidates by Pareto dominance
4. Crowding distance — measure how spread out solutions are on the Pareto front
5. Select, crossover, mutate — generate next generation
6. Repeat for N generations

```
from pymoo.algorithms.moo.nsga2 import NSGA2
from pymoo.optimize import minimize

result = minimize(
    NVHProblem(),
    NSGA2(pop_size=200),
    ('n_gen', 300),
    seed=42
)
# result.X → design variables of Pareto-optimal solutions
# result.F → [dBA, cost_index] for each solution
```

The Pareto Front

The **Pareto front** is the set of solutions where you cannot improve one objective without worsening another. Every point on the front is a valid "best" design — the choice between them is a business decision.





pareto_front.csv contains ~200 Pareto-optimal designs with their predicted dBA and cost_index values.

The Sweet Spot

The **engineering sweet spot** is the single recommended design point selected from the Pareto front using an elbow/knee criterion — where further cost increases yield diminishing noise reduction:

Parameter	Sweet-Spot Value
Fan blades	33.6 (≈ 34)
Fan RPM	2703
Panel damping	3.07 %
Sound pack density	23.0 kg/m³
Magnetic shield mass	0.15 kg
Air gap	12.0 mm
Isolator stiffness	8000 N/m
Predicted dBA	46.23 dBA
dBA reduction	13.77 dB

A 13.77 dBA reduction means the oven sounds approximately **5× quieter** to human perception (every 10 dB is perceived as roughly halving loudness).

9. Phase 5 — Sound Package Optimisation

What is a Sound Package?

A **sound pack** (or acoustic treatment package) is a collection of materials applied to oven panels to block and absorb noise. Examples include: - Melamine foam (absorber, NRC = 0.90) - Fiberglass batt (high-temperature absorber) - Mass Loaded Vinyl (heavy barrier, transmission loss via mass law) - Rubber damping mat (constrained-layer damping) - Air gap (impedance mismatch decoupling)

Mass Law

The primary physics governing barrier materials is the **mass law**:

$$IL(f) = 20 \cdot \log_{10}(m \cdot f \cdot \pi / (\rho \cdot c))$$

where:

- IL = Insertion Loss (dB) – how much the barrier blocks
- m = surface density of material (kg/m²)
- f = frequency (Hz)
- ρc = 415 Pa·s/m (characteristic impedance of air at 20°C)

Heavier materials block more sound, especially at high frequencies. The mass law predicts a 6 dB increase in IL for each doubling of mass or frequency.

Zone-by-Zone Optimisation

The oven is divided into 6 zones, each with different thermal limits, area, budget, and weight allowances:

Zone	Area (m²)	T-limit (°C)	Primary Source
Top Panel	0.070	250	Convection fan
Side Panel L	0.060	150	Convection fan
Side Panel R	0.060	150	Cooling fan
Rear Wall	0.080	200	Magnetron
Bottom Panel	0.070	120	Transformer
Door Inner	0.040	180	Door seal

For each zone, a greedy optimisation selects the material that maximises A-weighted single-number IL within thermal, budget, and weight constraints.

System-Level IL

The overall system Insertion Loss is the area-weighted average across all zones:

$$\text{System IL}(f) = \Sigma(\text{area_zone} \times \text{IL_zone}(f)) / \Sigma(\text{area_zone})$$

Results: The optimised sound pack achieves mean SIL = 13.1 dBA across all frequency bands at a total cost of \$3.04 and weight of 0.608 kg.

10. Phase 6 — LangGraph Multi-Agent System

What is LangGraph?

LangGraph is a framework (built on top of LangChain) for building multi-agent systems as directed graphs. Each node in the graph is an agent; edges define the communication flow.

Agent Graph Topology



Shared State (TypedDict)

All agents share a typed state dictionary:

```
class NVHState(TypedDict):
    query: str # original user question
    intent: str # classified intent
    source_analysis: dict # output from SourceAgent
    path_analysis: dict # output from PathAgent
    mitigation_plan: dict # output from MitigationAgent
    final_report: str # output from ReporterAgent
    agent_trace: list # breadcrumb of agents visited
    error: str # error message if any
```

Agent Specialisations

Agent	Role	Data Used
Supervisor	Intent classification, routing	None — rule-based
Source Agent	Identifies dominant noise sources from SPR data	source_ranking.csv, spr_matrix.csv
Path Agent	Traces transmission paths, suggests structural fixes	path_contributions.csv, ntf_data.csv
Mitigation Agent	Recommends treatment strategies with cost estimates	mitigation_matrix.csv, soundpack_bom.csv
Reporter Agent	Synthesises a structured NVH engineering report	All outputs

Why Multi-Agent?

A single LLM struggles to reason about multiple specialised domains simultaneously. By assigning each concern to a dedicated agent with its own context window, the system produces: - Higher quality domain-specific reasoning - Transparent reasoning traces (agent_trace) - Easy extensibility (add a new agent node without touching others)

11. Phase 7 — FastAPI REST Backend

Why FastAPI?

FastAPI provides: - Automatic OpenAPI docs (/docs endpoint, Swagger UI) - Pydantic validation — type-safe request/response schemas - Async support — non-blocking I/O for concurrent requests - CORS middleware — enables frontend/backend separation

API Endpoints (15 total)

Method	Endpoint	Description
GET	/	Health check + version
GET	/sources	Ranked noise source summary
GET	/paths	TPA path contributions

Method	Endpoint	Description
GET	/spr	SPR contribution matrix
GET	/mitigation	Mitigation strategy matrix
GET	/pareto	NSGA-II Pareto front data
GET	/sweet-spot	Engineering sweet-spot design parameters
GET	/soundpack	Sound pack BoM + system IL per frequency
GET	/competitors	Competitive benchmark data
GET	/reduction-estimate	Budget-optimised reduction estimate
POST	/agent/query	LangGraph multi-agent query
POST	/rag/query	BM25 RAG knowledge retrieval
GET	/doe	DOE dataset (paginated)
GET	/sensitivity	Sobol + SHAP sensitivity indices
GET	/surrogate-metrics	RF vs XGBoost model accuracy metrics

Running the API

```
cd nvh_oven_dashboard
uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload
# Swagger UI: http://localhost:8000/docs
```

Example Request

```
curl http://localhost:8000/sweet-spot
# Returns:
{
  "fan_blades": 33.56,
  "fan_rpm": 2703.11,
  "dba_pred": 46.23,
  "dba_reduction": 13.77,
  ...
}
```

12. Phase 8 — Interactive Streamlit Dashboard

Dashboard Overview

The dashboard has **7 tabs**, each visualising a different phase of the analysis:

Tab 1 — Noise Decomposition

- KPI cards: baseline dBA, top source, dominant path, A-weighted SPL
- 3D surface plot of source × path × contribution
- 1/3-octave spectra overlaid for all noise sources

- Ranked bar chart of sources by dBA contribution

Tab 2 — DOE Explorer

- Scatter plot matrix of all 7 design variables vs dBA response
- Parallel coordinates plot — visualise all 261 experiments simultaneously
- Feature correlation heatmap

Tab 3 — Surrogate Model & Sensitivity

- RF vs XGBoost performance comparison (R^2 , RMSE)
- SHAP feature importance bar chart
- Sobol total-order indices bar chart
- Prediction accuracy scatter (actual vs predicted)

Tab 4 — Optimisation (Pareto Front)

- Interactive Pareto front scatter plot (dBA vs cost_index)
- Sweet-spot design parameters table
- Hover tooltips showing all 7 design variables per solution

Tab 5 — Sound Pack BoM

- Zone-by-zone material selection table
- System IL improvement chart (by frequency band)
- Total cost, weight, and mean SIL summary metrics

Tab 6 — Competitive Intelligence

- Competitor product comparison table (measured dBA, price, fan type)
- Market positioning scatter (dBA vs price)
- IEC 60704 compliance status

Tab 7 — AI Knowledge Assistant (RAG + GPT-4o-mini)

- Free-text query input
- BM25 retrieval from 3 knowledge base documents
- GPT-4o-mini generation using retrieved context
- Collapsible source chunks showing which documents were used

Technical Implementation

```
# Data loading with Streamlit caching
@st.cache_data
def load_pareto():
    return pd.read_csv(SIM / "pareto_front.csv")

# Cached BM25 index built once per session
@st.cache_resource
def build_bm25():
    chunks, meta = [], []
    for txt_file in KB.glob("*.txt"):
```

```

text = txt_file.read_text(encoding="utf-8")
# sliding window chunking
sentences = re.split(r'(?<=[.!?])\s+', text)
for i in range(0, len(sentences), 4):
    chunk = " ".join(sentences[i:i+4])
    chunks.append(chunk.lower().split())
    meta.append({"source": txt_file.name, "chunk": chunk})
return BM25kapi(chunks), meta

```

13. Phase 9 — RAG Knowledge Assistant (GPT-4o-mini)

What is RAG?

Retrieval-Augmented Generation (RAG) combines information retrieval with LLM generation:

1. Retrieve: find relevant passages from a knowledge base using keyword search (BM25)
2. Augment: add retrieved passages as context to the LLM prompt
3. Generate: LLM answers the question using both its training knowledge and the retrieved context

RAG prevents hallucination by grounding the LLM in actual documents rather than relying purely on parametric memory.

Knowledge Base

Three domain-specific documents are indexed:

File	Content
NVH_standards.txt	IEC 60704-1/2 test conditions, measurement protocols, dBA limits
SPR_methodology.txt	SPR/TPA analysis theory, path ranking methods, NTF measurements
Mitigation_library.txt	Acoustic treatment techniques, material properties, NRC values

BM25 Retrieval

BM25 (Best Match 25) is a probabilistic keyword matching algorithm — the industry standard for lexical retrieval before neural embeddings:

```

from rank_bm25 import BM25kapi

tokenised_corpus = [chunk.lower().split() for chunk in chunks]
bm25 = BM25kapi(tokenised_corpus)

query_tokens = user_question.lower().split()
scores = bm25.get_scores(query_tokens)
top_k = sorted(range(len(scores)), key=lambda i: scores[i], reverse=True)[:3]

```

GPT-4o-mini Generation

Retrieved chunks are passed as context to GPT-4o-mini:

```
from openai import OpenAI
client = OpenAI(api_key=api_key)

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are an expert NVH engineer..."},
        {"role": "user", "content": f"Context:\n{retrieved_chunks}\n\nQuestion: {question}"},
    ],
    temperature=0.3,
    max_tokens=600,
)
```

Temperature = 0.3 keeps responses factual and deterministic while allowing some fluency.

Graceful Degradation

If no API key is configured, the system falls back to showing raw BM25 chunks without LLM generation — the dashboard remains fully functional offline.

14. Results & Key Outcomes

Noise Reduction

Metric	Baseline	Optimised	Improvement
Overall dBA	60.0 dBA	46.2 dBA	–13.8 dB
Perceived loudness	Reference	~4.5× quieter	Significant
IEC 60704 compliance	No	Yes (< 50 dBA)	✓

ML Model Performance

Model	Target: dBA R²	Target: Cost R²
Random Forest	0.865	0.935
XGBoost	0.869	0.959

Sound Pack Results

Metric	Value
Mean A-weighted SIL	13.1 dB
Total material cost	\$3.04
Added weight	0.608 kg
Zones treated	6 of 6

Key Engineering Insights

1. Fan blade count is the dominant noise driver (SHAP = 0.89, Sobol ST = 0.43) — changing from ~9 blades to ~34 blades shifts the Blade Passage Frequency (BPF) out of the sensitive hearing range and reduces tonal amplitude
 2. Fan RPM is the second most important factor (Sobol ST = 0.31) — reducing from 3500 RPM to 2703 RPM trades slight airflow reduction for 4–5 dBA improvement
 3. Panel damping above 2% provides diminishing returns — the optimal value (3.07%) balances IL improvement against cost
 4. Fiberglass Batt is optimal for all 6 zones due to its high-temperature rating (350°C), high NRC (0.95), and good IL across all frequencies
-

15. Project File Structure

```

NVH_Dashboard_StreamlitCloud_Deploy/
■
■■■■ requirements.txt           ← Streamlit Cloud lean requirements (8 packages)
■■■■ .gitignore                 ← Excludes API keys, generated files
■■■■ TECHNICAL_REPORT.md       ← This document
■
■■■■ nvh_oven_dashboard/
■
■■■■ app.py                     ← Streamlit Cloud entry point
■■■■ Dockerfile                 ← Multi-stage Docker build
■■■■ docker-compose.yml         ← Local stack (dashboard + API)
■■■■ requirements.txt           ← Full dev requirements
■■■■ packages.txt               ← System packages for Streamlit Cloud
■
■■■■ .streamlit/
■   ■■■■ config.toml           ← Theme, server config
■
■■■■ dashboard/
■   ■■■■ app.py                 ← Main 7-tab Streamlit dashboard
■
■■■■ data/                      ← All measurement & DOE data
■   ■■■■ source_spectra.parquet
■   ■■■■ doe_combined.csv / .parquet
■   ■■■■ sobol_indices.csv
■   ■■■■ shap_importance.csv (→ simulation/)
■   ■■■■ ...
■
■■■■ simulation/                 ← ML outputs & optimisation results
■   ■■■■ source_ranking.csv
■   ■■■■ spr_matrix.csv
■   ■■■■ path_contributions.csv
■   ■■■■ mitigation_matrix.csv
■   ■■■■ surrogate_metrics.csv
■   ■■■■ shap_importance.csv
■   ■■■■ pareto_front.csv
■   ■■■■ sweet_spot.json
■   ■■■■ soundpack_bom.csv
■   ■■■■ soundpack_summary.json
■   ■■■■ system_il_improvement.csv
■   ■■■■ noise_decomposition.py
■   ■■■■ soundpack_optimizer.py
■
■■■■ models/
■   ■■■■ surrogate_model.py     ← RF + XGBoost training & SHAP

```

```

■
■ ■ ■ agents/
■   ■ ■ ■ nvh_agents.py      ← LangGraph multi-agent system
■
■ ■ ■ api/
■   ■ ■ ■ main.py           ← FastAPI 15-endpoint backend
■
■ ■ ■ rag_engine/
■   ■ ■ ■ rag_pipeline.py    ← BM25 retrieval + GPT-4o-mini
■   ■ ■ ■ knowledge_base/
■       ■ ■ ■ NVH_standards.txt
■       ■ ■ ■ SPR_methodology.txt
■       ■ ■ ■ Mitigation_library.txt
■
■ ■ ■ competition/
■   ■ ■ ■ competitor_data.csv ← Benchmark product specs
■   ■ ■ ■ benchmark_analysis.py
■   ■ ■ ■ ...
■
■ ■ ■ reports/
■   ■ ■ ■ generate_report.py  ← Jinja2 + WeasyPrint PDF generator
■   ■ ■ ■ templates/
■       ■ ■ ■ nvh_report.html ← PDF report HTML template

```

16. Running the Project Locally

Prerequisites

- Python 3.10+
- Git

Setup

```

# 1. Clone the repository
git clone https://github.com/swapnil-kadlag/AI-Powered-NVH-Acoustic-Optimisation-Platform.git
cd AI-Powered-NVH-Acoustic-Optimisation-Platform

# 2. Create virtual environment
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate  # macOS/Linux

# 3. Install dependencies
pip install -r requirements.txt

# 4. (Optional) Add OpenAI API key for AI Chat
# Create nvh_oven_dashboard/env with content:
# OPENAI_API_KEY=sk-...

# 5. Run the dashboard
streamlit run nvh_oven_dashboard/app.py

```

The dashboard opens at <http://localhost:8501>.

Running the FastAPI Backend (optional)

```
cd nvh_oven_dashboard
```

```
pip install fastapi uvicorn pydantic langgraph langchain
uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload
# Swagger docs: http://localhost:8000/docs
```

17. Deploying to Streamlit Cloud

One-Time Setup

1. Push the repository to GitHub (must be public for free tier)
2. Go to share.streamlit.io
3. Select repository → set Main file path: `nvh_oven_dashboard/app.py`
4. Click Deploy

Adding the OpenAI API Key

In Streamlit Cloud: **Manage app** → **Settings** → **Secrets**

```
OPENAI_API_KEY = "sk-your-key-here"
```

How the Entry Point Works

Streamlit Cloud runs from the repository root. The entry point `nvh_oven_dashboard/app.py` uses `compile()` to execute the main dashboard while overriding `__file__` so all relative paths resolve correctly:

```
import sys, os
sys.path.insert(0, os.path.dirname(__file__))
_dashboard = os.path.join(os.path.dirname(os.path.abspath(__file__)), "dashboard", "app.py")
exec(compile(open(_dashboard, encoding="utf-8").read(), _dashboard, "exec"),
      {**globals(), "__file__": _dashboard})
```

18. Docker Deployment

A multi-stage Dockerfile is included for containerised deployment.

Build Stages

Stage 1 — Builder: installs system compilers and Python packages with C extensions

Stage 2 — Runtime: copies only the installed packages into a lean image

```
FROM python:3.11-slim AS builder
RUN apt-get install -y gcc g++ libpq-dev ...
COPY requirements.txt .
RUN pip install --prefix=/install -r requirements.txt

FROM python:3.11-slim
COPY --from=builder /install /usr/local
COPY nvh_oven_dashboard /app
CMD ["streamlit", "run", "app.py", "--server.port=8501"]
```

Running with docker-compose

```
docker-compose up --build
```

19. Key Engineering Concepts Explained

Blade Passage Frequency (BPF)

When a fan has B blades spinning at N RPM, it generates a tonal noise peak at:

$$BPF = (N / 60) \times B \quad [Hz]$$

Example: 9 blades at 3500 RPM → BPF = 525 Hz (annoying, A-weight ~ 0 dB)
34 blades at 2703 RPM → BPF = 1530 Hz (less sensitive, easier to treat)

Shifting BPF to a frequency where A-weighting correction is smaller reduces perceived noise even if the absolute sound power stays the same.

Decibel (dB) Scale

Sound pressure level is logarithmic:

$$SPL = 20 \cdot \log_{10}(p / p_{ref}) \quad [dB]$$

$p_{ref} = 20 \mu Pa$ (threshold of human hearing)

Key rules: - **+10 dB** ≈ twice as loud (perceived) - **+6 dB** = double the sound pressure - **+3 dB** = double the acoustic power - Combining two equal sources: $SPL_{total} = SPL + 3 \text{ dB}$ (not 2×)

Insertion Loss vs Sound Absorption

Concept	What it does	Unit	Measured by
Insertion Loss (IL)	Blocks sound from passing through a barrier	dB	Comparing levels with/without barrier
Noise Reduction Coefficient (NRC)	Absorbs sound within a room/cavity	0–1	Reverberation room test
Transmission Loss (TL)	Same as IL but measured in a specific lab setup	dB	ISO 10140

Sound packages need both: **absorbers** (foam, fiberglass) inside the oven cavity reduce reverberant build-up; **barriers** (MLV, steel panels) block noise radiating outward.

Pareto Dominance

Solution A **dominates** solution B if: - A is at least as good as B in all objectives - A is strictly better than B in at least one objective

The **Pareto front** is the set of all non-dominated solutions. In engineering, you always want to operate on the Pareto front — any solution off the front is wasteful (a dominated solution can be improved in at least one objective at no cost to others).

BM25 vs Embeddings

Method	How it works	Strength	Weakness
BM25	Term frequency × inverse document frequency	Fast, no GPU, exact keyword match	Misses synonyms, no semantic understanding
Dense embeddings (e.g., text-embedding-3-small)	Neural network encodes semantic meaning	Handles synonyms, conceptual queries	Needs embedding API/GPU, more complex

For a technical domain (NVH, IEC standards) with precise terminology, BM25 performs well — engineers use specific terms consistently.

Acknowledgements

This platform is built using open-source tools and draws on the following standards and methodologies:

- IEC 60704-1:2010 — Household electrical appliances — Test code for the determination of airborne acoustical noise
 - SALib — Sensitivity Analysis Library (Herman & Usher, 2017)
 - pymoo — Multi-objective Optimisation in Python (Blank & Deb, 2020)
 - LangGraph — Agent framework by LangChain, Inc.
 - SHAP — A unified approach to interpreting model predictions (Lundberg & Lee, 2017)
-

This project demonstrates the application of data science, machine learning, and generative AI to a real-world product engineering challenge — from measurement data to a live deployed dashboard with an AI assistant.